```
def getCargos(in: EntityId): Future[Cargos]
```

It is almost the same as the previous Akka HTTP example, except that there are inconsistencies between the objects in the response. You get an actor message type, but you need a .proto message type. You need to convert `the` actor `Cargos` to `.proto` `Cargos`. You can do this by mapping the response from `ask` to `Cargos` with `.mapTo[Container.Cargos]` and then map over the cargos to convert them to an `example.container.grpc.Cargo`.

Listing 11.9 `getCargos` implementation

```
class ContainerServiceImplSharding(sharding: ClusterSharding)(
  ...

  implicit val timeout: Timeout = 3.seconds                ❶

  override def getCargos(in: EntityId): Future[Cargos] = {
    val container =
      sharding.entityRefFor(Container.TypeKey, in.entityId)

    container
      .ask(Container.GetCargos)
      .mapTo[Container.Cargos]
      .map { containerCargos =>
        val c = containerCargos.cargos.map { each =>       ❷
          Cargo(each.kind, each.size)                      ❷
        }
        Cargos(c)                                          ❸
      }
  }
}
```

❶ Timeout for .ask

❷ Each Container.Cargo is transformed to a grpc.Cargo.

❸ All the grpc.Cargos are used to build a grpc.Cargos.